
PPO for Autonomous Car Racing: A Comparative Study of Reward and Action Space Strategies

Chao Bian, Harvey Zhou, Michael Yang, Rui Tong, Yiheng Feng

Department of Computer Science
University of California, Santa Barbara
Santa Barbara, CA 93106
yang335@ucsb.edu
mukunzhou@ucsb.edu
yang335@ucsb.edu
ruitong@ucsb.edu
yihengfeng@ucsb.edu

Abstract

This course project explores five strategies for training an autonomous car using Proximal Policy Optimization (PPO) to complete a racing track as quickly as possible. The experiments examine variations in action space (discrete vs. continuous), reward schemes (discrete, continuous, and cumulative), and observation features (including visual input). Our findings demonstrate that our proposed cumulative rewards in continuous action-space settings are most effective for completing the track. We also present a mathematical relationship between the immediate and cumulative reward formulations used in PPO. The code can be found at <https://github.com/TantaiYanhen/CS190A>. We also have a demo.

1 Introduction and Related Work

Reinforcement Learning (RL) has emerged as a powerful tool for sequential decision-making tasks, demonstrating notable success in domains ranging from game playing [5, 8] to robotic manipulation [4]. A key challenge in RL is the design of algorithms that can learn effective policies in environments with high-dimensional state and action spaces, delayed rewards, and stochastic dynamics.

Proximal Policy Optimization (PPO), introduced by Schulman et al. [7], is a widely used on-policy algorithm that has become a popular choice in many continuous control problems due to its robust training performance and theoretical grounding. It optimizes a clipped surrogate objective that avoids large policy updates with a KL divergence term as regularization to stabilize the training process:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \right], \quad (1)$$

The versatility of PPO has led to its adoption in a variety of applications beyond classical control, including natural language generation [10], healthcare [9], and finance [1]. In the field of autonomous driving, PPO has been used to learn complex driving behaviors under diverse scenarios [3, 2], enabling the agent to adapt to dynamic environments and generalize across varying road conditions, traffic patterns, and sensor modalities.

In this work, we investigate how the structure of the action space, the design of the reward function, and the nature of observations impact the performance of an RL agent trained with PPO to drive an autonomous vehicle on a challenging, realistic track. Our goal is to identify the best design choice that produces the most efficient and reliable policy learning in the context of autonomous driving by evaluating the final policy performance under varying settings.

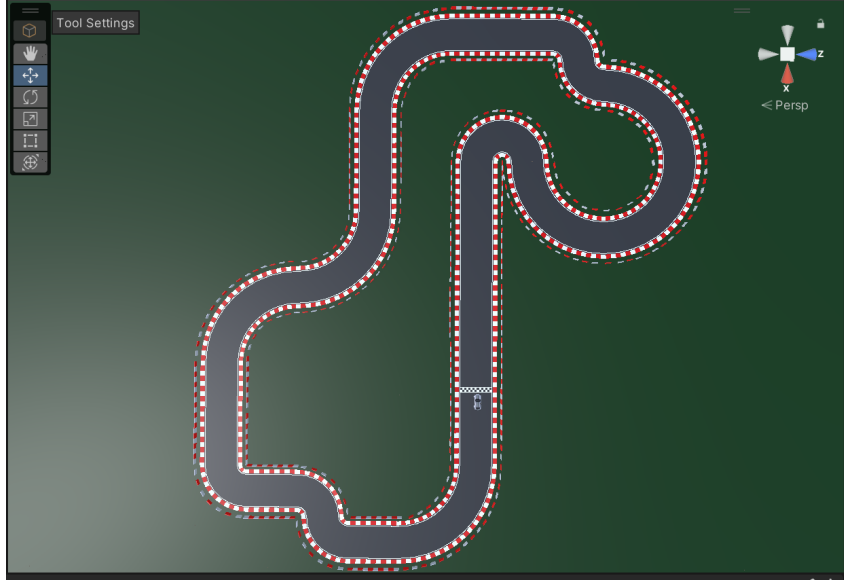


Figure 1: Training environment in Unity.

2 Methods

2.1 Action Space

We experimented with both discrete and continuous action spaces:

- **Discrete Action Space:** The action set is defined as:

$$\mathcal{A}_{\text{discrete}} = \{\text{Forward, Left-Forward, Right-Forward, Brake, Idle}\}$$

- **Continuous Action Space:** The car chooses from a continuous 2D action space:

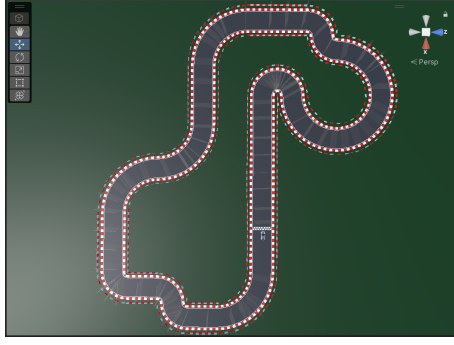
$$\mathcal{A}_{\text{continuous}} = [-1, 1]^2$$

where the first component controls forward/backward thrust and the second controls left/right steering.

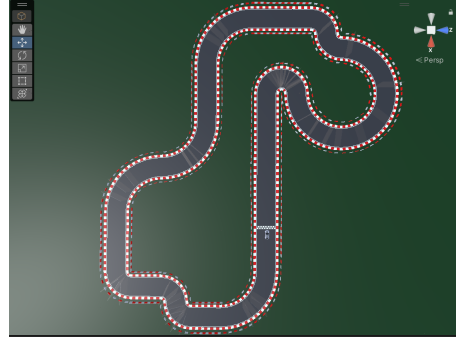
2.2 Observation Space

The observation space consists of both numerical and visual inputs designed to provide the reinforcement learning (RL) agent with sufficient information to make informed decisions while navigating the track. The components of the observation space are as follows:

1. **Position Coordinates** (x, y) : The global 2D coordinates of the car on the map, useful for determining the absolute location.
2. **Orientation**: The heading direction of the car, typically represented by an angle or a unit vector, indicating the car's facing direction relative to a fixed reference.
3. **Progress Along the Track** t : We define a reference spline that follows the left boundary of the track. At each timestep, the car's position is orthogonally projected onto this spline. The resulting scalar parameter t represents the normalized progress of the car along the spline (e.g., from $t = 0$ at the start to $t = 1$ at the end).
4. **Distance to Left Boundary**: Once the projection onto the spline is obtained, the shortest distance from the car to the spline is computed. This provides a continuous measure of how far the car is from the left boundary of the track.
5. **Top-Down View Image**: A simulated top-down camera captures an image of the area around the car. This visual input allows the agent to learn spatial patterns and anticipate upcoming turns or obstacles.

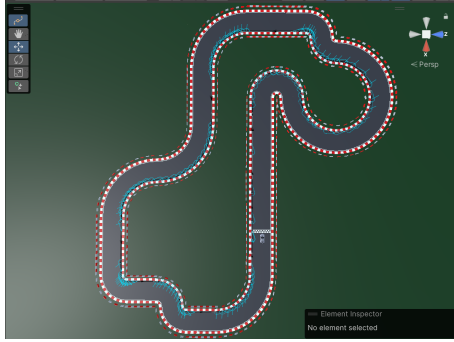


(a) All walls

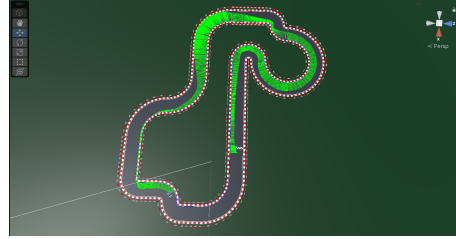


(b) High reward walls.

Figure 2: a) Walls along the track as discrete reward. b) The walls along the turns have higher reward, encouraging the car to focus on difficult parts of the task.



(a) All walls



(b) High reward walls.

Figure 3: a) Spline that sticks to the left boundary of the track. b) Green line segments represents the car's distance to the left boundary.

In different experimental settings, we enable different subsets of these observations to study their effects on learning efficiency and driving performance.

2.3 Reward Schemes

We used the following variations in reward:

1. **Discrete Reward, Time Penalty:** Positive reward from passing over “reward walls,” negative reward when hitting track boundaries. Each wall worth 10 points, and hitting the boundary gives -200 penalty. Also, the car gets 1 point penalty every second.
2. **Discrete Reward with Curve Bonus:** Additional rewards placed on curves ($\times 10$) to encourage turning.
3. **Continuous Linear Reward:** Reward proportional to progress; no discrete walls.

$$R_k = \Delta t_k * 100$$

4. **Cumulative Reward:** Define R_k as step reward and cumulative reward as:

$$R'_k = \sum_{i=1}^k R_i$$

If $\Delta t_k < 0$, the set reward to -1 .

5. **Vision-Based Reward:** Agent only receives RGB image as observation.

2.4 Theoretical analysis of Cumulative Reward

In this subsection we explore the behavior of cumulative reward function under different settings:

Proposition 1: The policy under cumulative reward converges to the policy under original reward as the number of timesteps used in value estimation increases to infinity.

Proof: The original value function is: $V(s_k) = \mathbb{E}_\pi [\sum_{i=0}^{\infty} \gamma^i R_{k+i} \mid s_k]$. If we use cumulative reward $R'_k = \sum_{j=1}^k R_j$, then:

$$\begin{aligned}
V'(s_k) &= \mathbb{E}_\pi \left[\sum_{i=0}^{\infty} \gamma^i R'_{k+i} \mid s_k \right] \\
&= \mathbb{E}_\pi \left[\sum_{i=0}^{\infty} \gamma^i \left(\sum_{j=0}^{k+i} R_j \right) \mid s_k \right] \\
&= \mathbb{E}_\pi \left[\sum_{j=0}^{\infty} R_j \sum_{i=\max(0, j-k)}^{\infty} \gamma^i \mid s_k \right] \\
&= \mathbb{E}_\pi \left[\sum_{j=k}^{\infty} R_j \sum_{i=j-k}^{\infty} \gamma^i \mid s_k \right] \\
&= \mathbb{E}_\pi \left[\sum_{j=k}^{\infty} R_j \cdot \frac{\gamma^{j-k}}{1-\gamma} \mid s_k \right] \\
&= \frac{1}{1-\gamma} \mathbb{E}_\pi \left[\sum_{j=k}^{\infty} \gamma^{j-k} R_j \mid s_k \right] \\
&= \frac{1}{1-\gamma} \mathbb{E}_\pi \left[\sum_{i=0}^{\infty} \gamma^i R_{k+i} \mid s_k \right] \\
&= \frac{1}{1-\gamma} V(s_k)
\end{aligned}$$

Thus, cumulative reward shifts the value function by a factor of $\frac{1}{1-\gamma}$ as the number of timesteps increase to infinity. However, in practice, we won't use so many timesteps, resulting a different behavior of policy model.

2.5 Environment and Termination

The car resets to origin if it touches the boundary or the length of current epoch exceeds 2 minutes. Death and time penalty are applied and will be elaborated in Section 3.

3 Results

3.1 Experiment Setup

We use the built-in implementation of Proximal Policy Optimization (PPO) from the Stable-Baselines3 library [6]. The PPO agent is trained in a custom 3D car racing environment we built in Unity. Communication between the Unity engine and the Python training loop is established via gRPC, with python end as the server and unity as the client, enabling real-time data exchange and synchronization between the simulation and the learning algorithm.

The Unity environment includes various turns and straight segments to encourage complex driving behavior. A *spline* is embedded along the left boundary of the track. During training, the agent's state includes its projection onto this spline, allowing us to compute both:

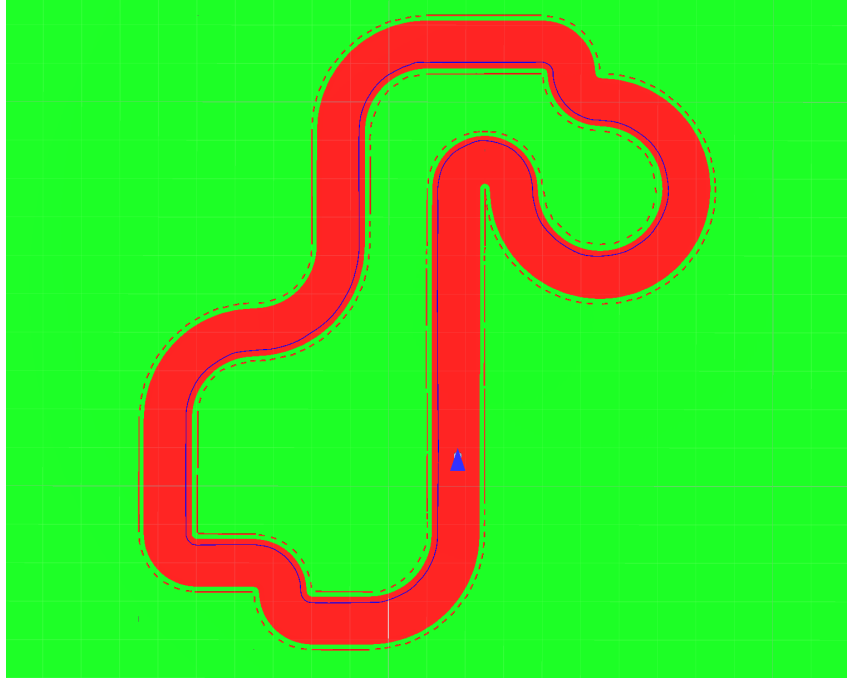


Figure 4: Visual RGB from the Unity environment used as visual observation. The original texture is repainted to modify the coloring of the whole scene. A triangle is placed above the car as indication of orientation.

- **Progress** along the track: measured as the arc length $0 \leq t \leq 1$ of the closest projection point onto the spline.
- **Distance to left boundary**: represented by the Euclidean distance between the agent's center and the nearest point on the spline.

For experiments involving visual inputs, the track scene is rendered with no textures and high visual contrast to facilitate feature extraction. Specifically, the car, the background, and the road are each assigned unique, solid colors to minimize visual ambiguity and increase the agent's ability to distinguish between relevant objects.

3.2 Ablation Study

Method 1: Discrete reward with time penalty

In this most simple basic setting, the model fails to pass the first turn. It keeps trying to turn right, but because of the high speed it obtained after passing the first few blocks on the straight line, it is unable to make a successful turn. This result implies that the turning reward is too low since the agent satisfied itself with the rewards given by the blocks on the straight line.

Method 2: Discrete reward with curve bonus

To encourage the model to make turns, we increased the reward of the walls around the turning point by 10 times. Rewarding curves improves the car's turning ability significantly. After the reward is increased, the car tends to turn right at the end of the straight line. We also explored the relationship between the car's performance and the number of walls on the track. With more curve reward walls, the agent eventually makes turns, but still fails to complete the full track. This hinted us to make infinite number of walls.

Method 3: Add observation and continuous reward

Hinted by the previous trial, we decided to make the reward be continuous with respect to time, which mimics putting infinite number of walls on the track. What's more, to make the car be aware of its distance to the boundary, we added this value in the observation space, which allows better learning

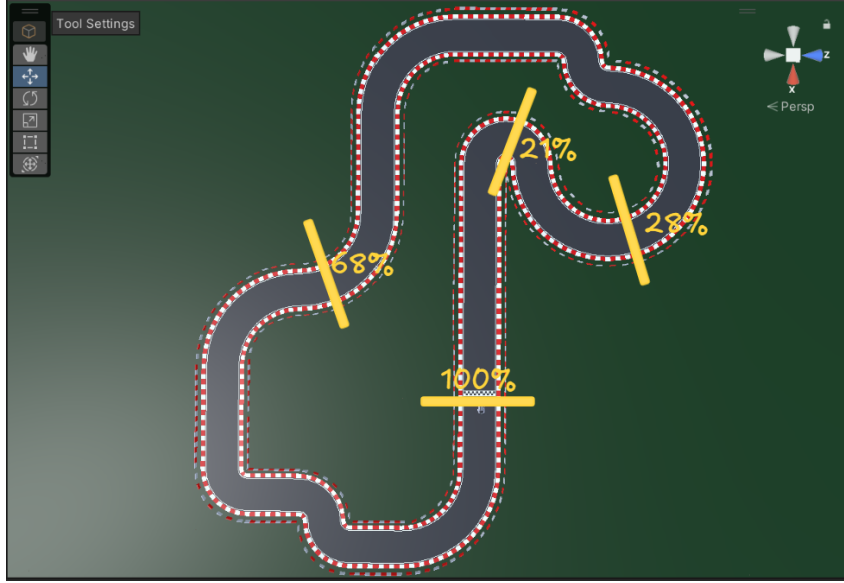


Figure 5: Completion of each method.

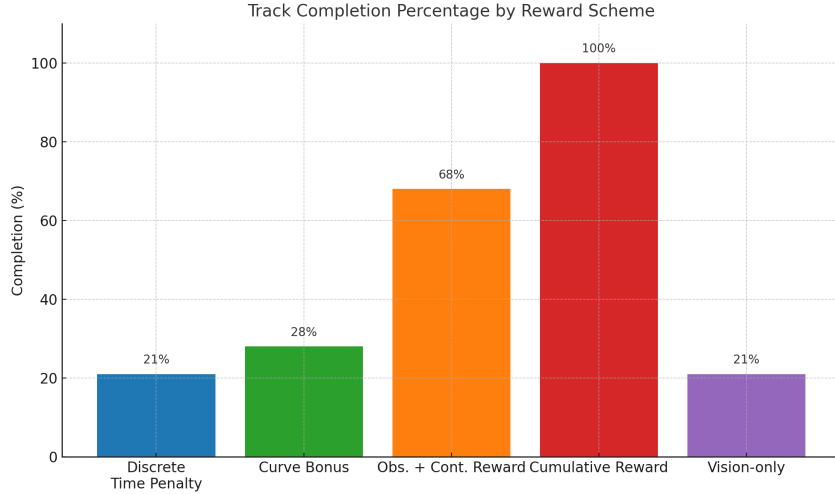


Figure 6: Completion of each method.

and much higher completion ratio of the task. However, the improvement is very slow and the agent still fails to complete the entire track.

Method 4: Cumulative reward

To overcome the local stagnation of previous methods, we used our proposed cumulative reward function. This approach shifts the optimization goal to maximizing long-term accumulated reward rather than short-term gain. Empirically, this modification led to the most successful results. The agent learnt to complete the full track, executing both straight-line acceleration and precise turning behaviors.

Method 5: Vision-only observation

We also explored how the visual information can help the car learn its position. We feed a 64×64 image with the car in the center as visual input to a CNN model. Despite altering the background, car, and track colors for clarity, the model fails to learn a policy that finishes the track, implying that more advanced methods like UNet might necessary to improve the performance.

4 Conclusion

We conducted a systematic evaluation of PPO training strategies for autonomous car navigation. Our results show that simple discrete rewards are insufficient for complex behavior like turning. Adding curve bonuses and continuous features improves performance, but cumulative reward formulation is the key to long-term successful driving. Vision-only models require additional training and architectural tuning.

5 Contribution Statement

Mukun Zhou: Wrapped Unity environment into OpenAI Gymnasium environment, refined Python-Unity communication protocol, defined rewarding policy, trained and evaluated models with different hyperparameters.

Michael Yang: Construct basic setup of connection between Python server and Unity client with GRPC. Write report. Implementation of visual observation part in experiment.

Rui Tong: I changed the input from discrete controls (forward/backward/right/left at constant speed) to a continuous 2D vector, improving the car’s performance and allowing it to turn smoothly. I also tried to write different reward functions and give them weights to avoid overfitting.

Chao Bian: Construct the Unity race car track environment. Implementation of progrss-based reward and collision penalty.

Yiheng Feng: Resolved physics inconsistencies by reconfiguring Fixed Delta Time and decision frequency. Implementation of progrss-based reward and collision penalty.

References

- [1] Arthur Charpentier, Romuald Elie, and Carl Remlinger. Reinforcement learning in economics and finance. *Computational Economics*, pages 1–38, 2021.
- [2] Yinbo Chen, Sifei Liu, and Xiaolong Wang. Learning continuous image representation with local implicit image function. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 8628–8638, 2021.
- [3] Alexey Dosovitskiy, German Ros, Felipe Codevilla, Antonio Lopez, and Vladlen Koltun. Carla: An open urban driving simulator. In *Conference on robot learning*, pages 1–16. PMLR, 2017.
- [4] Deepak Gopinath, Xiongyi Cui, Jonathan DeCastro, Emily Sumner, Jean Costa, Hiroshi Yasuda, Allison Morgan, Laporsha Dees, Sheryl Chau, John Leonard, et al. Computational teaching for driving via multi-task imitation learning. *arXiv preprint arXiv:2410.01608*, 2024.
- [5] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.
- [6] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Noah Ernestus, and Maximilian Dormann. Stable-baselines3: Reliable reinforcement learning implementations. <https://github.com/DLR-RM/stable-baselines3>, 2021.
- [7] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [8] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587):484–489, 2016.
- [9] Chao Yu, Jiming Liu, Shamim Nemati, and Guosheng Yin. Reinforcement learning in healthcare: A survey. *ACM Computing Surveys (CSUR)*, 55(1):1–36, 2021.

- [10] Daniel M Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*, 2019.